Retrieval-Augmented Generation (RAG)

Comprehensive Course Curriculum

From Fundamentals to Production-Ready Agentic RAG Systems

Course Overview

This curriculum provides a structured, end-to-end path through Retrieval-Augmented Generation — beginning with core concepts and document processing, advancing through embeddings, vector stores, and retrieval techniques, and culminating in agentic RAG systems built with LangGraph and a deployable capstone project.

Module Map

- Module 1 — RAG Fundamentals and Architecture
- Module 2 — Document Processing and Chunking
- Module 3 — Embeddings and Vector Representations
- Module 4 — Vector Stores
- Module 5 — Basic Retrieval Techniques
- Module 6 — Advanced Retrieval Techniques
- Module 7 — Advanced RAG Patterns
- Module 8 — Agentic RAG with LangGraph
- Module 9 — Evaluating RAG
- Module 10 — Capstone Project with Deployment

Prerequisites

1. Python fundamentals
2. GenAI fundamentals
3. LangChain
4. LangGraph

Module 1: RAG Fundamentals and Architecture

1.1 Introduction to RAG

1.1.1 What is RAG and the Problems It Solves

- Hallucination reduction
- Knowledge cutoff limitations
- Domain-specific knowledge injection
- Source attribution and transparency

1.1.2 Core Components

- Knowledge Base → Retriever → Generator

1.1.3 RAG Data Flow

- End-to-end flow from user query through retrieval to generation

1.2 RAG vs Fine-Tuning vs Prompt Engineering

- Comparison matrix: cost, latency, accuracy, maintenance
- Decision framework: when to use each approach

Module 2: Document Processing and Chunking

2.1 Document Loaders in LangChain

- PDF loaders: PyPDFLoader, PDFMiner, PDFPlumber, Unstructured
- Web loaders: WebBaseLoader, RecursiveUrlLoader
- Structured data: CSV, JSON
- Text Documents: Textloader
- Hands-on: Load documents from 5 different sources

2.2 Text Splitting Strategies

- RecursiveCharacterTextSplitter (recommended default)
    - Hierarchical separator logic
    - Parameter tuning: chunk_size, chunk_overlap
- CharacterTextSplitter and TokenTextSplitter
- Semantic chunking with SemanticChunker
    - Breakpoint threshold types: percentile, standard_deviation, gradient
- MarkdownTextSplitter for structured documents
- Code splitters for Python, JavaScript, etc.
- LLM Based Text Splitting
- Hands-on: Compare chunking strategies on the same document

2.3 Chunking Best Practices

- Optimal chunk sizes by use case
    - Factoid queries: 128–256 tokens
    - General RAG: 256–512 tokens
    - Complex analysis: 512–1024 tokens
- Overlap strategies (10–20% recommended)
- How chunk size affects retrieval quality
- Handling tables and structured data
- Preserving context across chunks
- Lab: Optimize chunking for a PDF technical document

2.4 Metadata Management

- Adding metadata to documents
- Metadata filtering in retrieval
- Source tracking and document lineage
- Document versioning strategies
- Hands-on: Build a metadata-enriched document pipeline

Module 3: Embeddings and Vector Representations

3.1 How Embeddings Work

- Vector representations of text
- Semantic similarity in embedding space
- Distance metrics: cosine similarity, Euclidean, dot product
- Embedding dimensions and their trade-offs

3.2 Embedding Models Overview

- OpenAI embeddings
    - text-embedding-3-small ($0.02 / 1M tokens) — best value
    - text-embedding-3-large ($0.13 / 1M tokens) — highest quality
- Open-source alternatives
    - Embedding Gemma through Ollama python lib
    - Embedding Gemma through langchain-ollama

3.3 Choosing the Right Embedding Model

- MTEB leaderboard and what to look for
- Dimension trade-offs: 384 → 768 → 1536 → 3072
- Cost vs quality decision framework
- Hands-on: Compare embedding models on sample queries

3.4 LangChain Embeddings Implementation

- Embeddings class usage
- embed_documents vs embed_query

Module 4: Vector Stores

4.1 Vector Store Comparison and Selection

Free Vector Store Landscape

Store	Type	Best For	Free Tier
Chroma	Embedded	Learning, prototyping	Unlimited (self-hosted)
FAISS	Library	High-performance search	Unlimited
Qdrant	Server	Production	1GB cloud forever
Pinecone	Managed	Zero-ops deployment	100K vectors
Milvus	Server	Billion-scale	Unlimited (self-hosted)

4.2 Vector Store Operations (CRUD)

- Create: Adding documents and texts
- Read: Similarity search, search with scores
- Update: Modifying existing documents
- Delete: Removing documents by ID or filter
- Lab: Build a full CRUD vector store application

Module 5: Basic Retrieval Techniques

5.1 Similarity Search Fundamentals

- How vector similarity search works
- similarity_search() method
- Search with relevance scores
- Configuring k and search parameters
- Hands-on: Basic similarity search implementation

5.2 Similarity Score Threshold

- Why thresholds matter for quality control
- Setting appropriate thresholds
- LangChain similarity_score_threshold search type
- Hands-on: Implement threshold-based retrieval

5.3 MMR (Maximal Marginal Relevance)

- Balancing relevance and diversity
- Lambda parameter tuning (0 = diversity, 1 = relevance)
- fetch_k vs k parameters
- Use cases: summarization, complex queries
- Hands-on: Compare MMR vs standard similarity search

5.4 Hybrid Search

- Dense vectors (semantic) + sparse vectors
- When hybrid search outperforms pure semantic
- BM25Retriever in LangChain
- Weighting strategies (alpha parameter)
- Hands-on: Build hybrid search with EnsembleRetriever
- Lab: Compare semantic vs keyword vs hybrid on docs

Module 6: Advanced Retrieval Techniques

6.1 Contextual Compression

- Why compress retrieved context

- LLMChainExtractor: LLM-based extraction
- EmbeddingsFilter: Fast embedding-based filtering
- DocumentCompressorPipeline: Chaining compressors
- Hands-on: Implement a contextual compression pipeline

6.2 Parent Document Retriever

- The chunk-size dilemma: small for matching, large for context
- Child chunks for retrieval, parent chunks for context
- InMemoryStore and docstore configuration
- Hands-on: Build a parent-child document system

6.3 Self-Query Retriever

- Natural language to structured queries
- AttributeInfo metadata schema definition
- Automatic filter extraction
- Hands-on: Product catalog search with self-query

6.4 Multi-Query Retriever

- Query expansion using LLM
- Improving recall through query variations
- Hands-on: Implement multi-query retrieval

6.5 Re-ranking Strategies

- Two-stage retrieval: retrieve → re-rank (RAG Fusion re-ranking covered)
- Cross-encoder re-rankers
- Cohere Rerank integration
- Hands-on: Add re-ranking to an existing pipeline

Module 7: Advanced RAG Patterns

7.1 RAG Fusion

- Multiple query generation + Reciprocal Rank Fusion
- Workflow and architecture
- Trade-offs: comprehensiveness vs latency
- Hands-on: Build a RAG Fusion pipeline

7.2 HyDE (Hypothetical Document Embeddings)

- Generate hypothetical answer → embed → search
- Bridging the query–document semantic gap
- Hands-on: Implement HyDE with LangChain

7.3 Corrective RAG (CRAG)

- Document relevance grading
- Web search fallback mechanism
- Query rewriting when retrieval fails
- Hands-on: Build CRAG with LangGraph

7.4 Self-RAG

- Self-RAG: Reflection tokens and self-correction
- Query complexity classification

7.5 Graph RAG

- Knowledge graphs + RAG
- Neo4J GraphRAG approach
- Adding Data to Graph Database using pdf documents.
- Working with Multi Hop Queries
- When Graph RAG outperforms vector RAG

7.6 Multi-Modal RAG (to be added soon)

- Extending RAG to images, tables, and mixed-content documents

---

Module 8: Agentic RAG with LangGraph

8.1 Introduction to Agentic RAG

- What is Agentic RAG and why it matters
- Traditional RAG vs Agentic RAG comparison
- Key capabilities: reasoning, tool use, multi-step execution

- Market significance and real-world impact

8.2 RAG as a Tool for Agents

- Creating retriever tools with create_retriever_tool
- When agents should retrieve vs respond directly
- Multiple knowledge base tools
- Hands-on: Build an agent with a retrieval tool

8.3 LangGraph Fundamentals for RAG

- State, nodes, edges, and conditional routing
- State schemas for RAG (TypedDict)
- Building RAG graphs step-by-step
- Hands-on: Build an agentic RAG graph from scratch

8.4 Agentic RAG Design Patterns

- ReAct pattern with RAG
- Plan-and-execute with retrieval
- Reflection / self-correction patterns
- Hands-on: Implement a ReAct RAG agent

Module 9: RAG Evaluation using RAGAS

9.1 RAG Evaluation

- Key metrics: faithfulness, relevance, context precision, context recall, noise sensitivity
- RAGAS framework implementation
- Creating evaluation datasets
- Lab: Evaluate each metric individually and evaluate API on the RAG pipeline.

Module 10: Capstone Project with Deployment [To be added soon]

10.1 Project Assignment: Build a Production-Ready RAG System

- Multi-document ingestion pipeline
- Hybrid retrieval with re-ranking
- Agentic workflow with LangGraph
- Evaluation suite with RAGAS
- Monitoring with LangSmith
- Deployment (TBD)

10.2 Production RAG Good Practices

- Pipeline Optimization
- Caching Strategies
- Cost Optimization
- Monitoring & Debugging
- Common Pitfall and Solutions
- Security and Compliances